Indian Institute of Technology Hyderabad
Department of Computer Science and Engineering

Programming Assignment 2

DNS and DNSSEC

CS6903: Network Security, 2025–26

Name : *Atish Kadam*
Roll No. : *CS25MTECH14003*
Date : 22nd April 2026

Submitted to: Department of CSE, IIT Hyderabad

Anti-Plagiarism Statement

We certify that this assignment/report is our own work, based on our personal study and/or research, and that we have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication. We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand our responsibility to report honor violations by other students if we become aware of them.

Name ⟨roll numbers⟩: Atish Kadam (`CS25MTECH14003`)

Date: 22nd April 2026

Signature: Atish Kadam

Contents

1. Introduction and Project Overview

This report presents a DNSSEC toolkit developed for Programming Assignment 2 of CS6903 Network Security. The toolkit validates DNS data using signatures, trust anchors, delegation records, and denial-of-existence proofs.

1.1 Project Structure

File	Description
`q1_dnssec_validator.py`	Core DNSSEC validation library (imported by all others)
`q2_resolver.py`	Recursive resolver with per-hop DNSSEC validation
`q3_denial.py`	Authenticated denial of existence (NSEC/NSEC3)
`q4_lifecycle.py`	Key lifecycle and rollover analysis
`q5_tamper.py`	Tamper simulation and detection (SEED Lab)

1.2 System Requirements

```
pip install dnspython
```

All modules depend on `q1_dnssec_validator.py`. Run it first to confirm that the core validation logic works correctly.

2. Task 1 — Core DNSSEC Validator (`q1_dnssec_validator.py`)

2.1 Description

This task implements the core DNSSEC validation flow. It retrieves the answer record, DNSKEY, and DS records, then verifies the RRSIG and the parent-to-child chain of trust.

The validator checks whether the DNS answer is cryptographically authentic by matching the answer RRSIG against the zone DNSKEY and confirming the DNSKEY against the parent DS record.

2.2 Command to Run

```
python3 q1_dnssec_validator.py <domain> <record_type>

# Examples
python3 q1_dnssec_validator.py cloudflare.com A
python3 q1_dnssec_validator.py cloudflare.com AAAA
python3 q1_dnssec_validator.py google.com MX
python3 q1_dnssec_validator.py example.com A
```

2.3 Screenshots

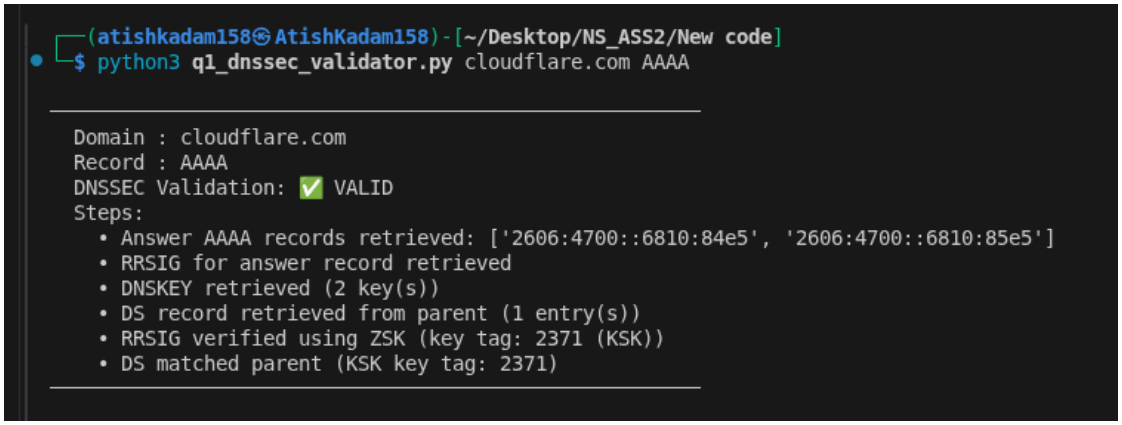


Figure 1: Task 1 — Output for `cloudflare.com AAAA` (DNSSEC VALID)

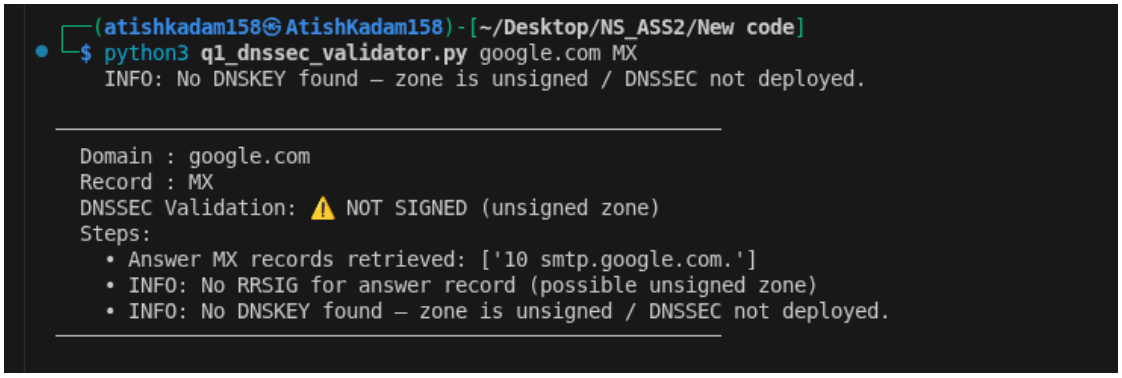


Figure 2: Task 1 — Output for `google.com MX`

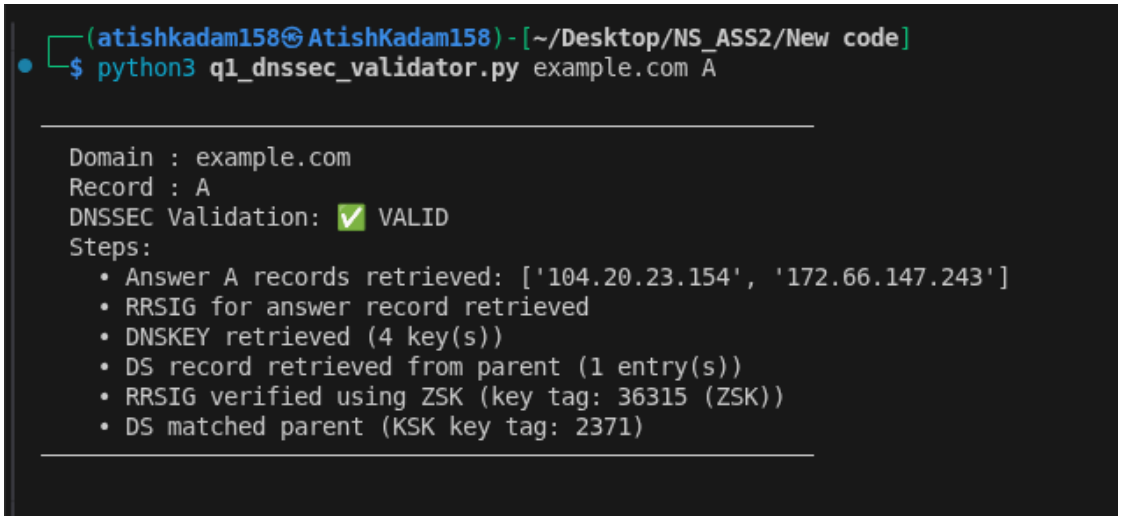


Figure 3: Task 1 — Output for `example.com A`

3. Task 2 — Recursive Resolver (`q2_resolver.py`)

3.1 Description

This task builds an iterative resolver that follows the DNS delegation path from the root to the authoritative server while validating DNSSEC at each hop.

The resolver queries each delegation step directly and checks the returned DNSSEC data locally instead of trusting an upstream recursive resolver.

3.2 Command to Run

```
python3 q2_resolver.py <domain> <record_type>

# Examples
python3 q2_resolver.py example.com A
python3 q2_resolver.py cloudflare.com AAAA
python3 q2_resolver.py ietf.org MX
```

3.3 Screenshots

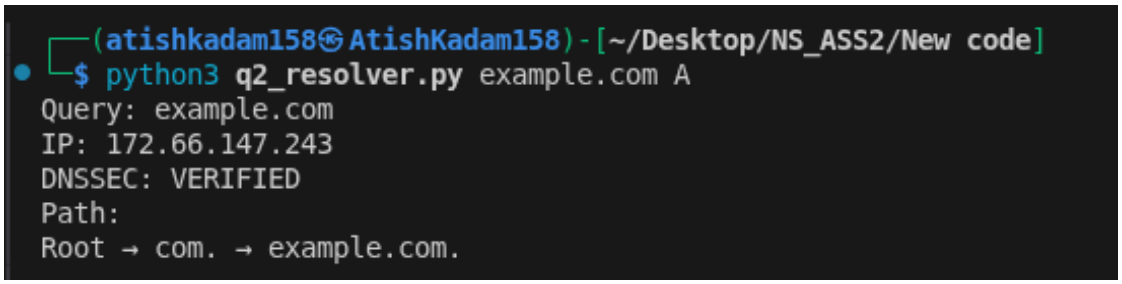


Figure 4: Task 2 — Recursive resolution of `example.com A`

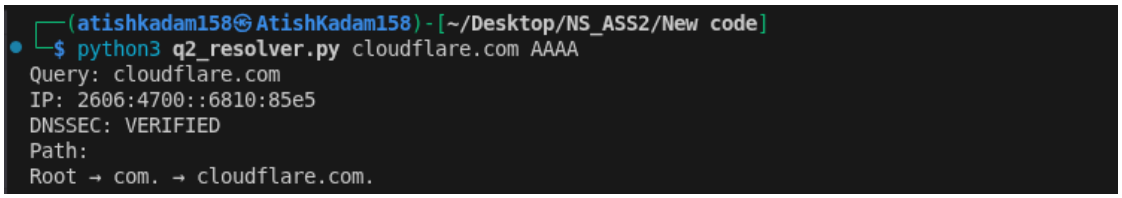


Figure 5: Task 2 — Recursive resolution of `cloudflare.com AAAA`

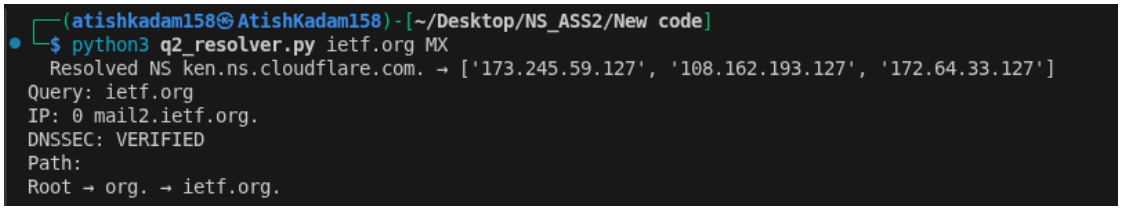


Figure 6: Task 2 — Recursive resolution of `ietf.org MX`

4. Task 3 — Authenticated Denial of Existence (`q3_denial.py`)

4.1 Description

This task verifies that a missing domain name or missing record type can be proved securely using NSEC or NSEC3 records.

The module distinguishes between NXDOMAIN, NODATA, and EXISTS, and validates the denial proof with DNSSEC signatures.

4.2 Command to Run

```
python3 q3_denial.py <domain> <record_type>

python3 q3_denial.py thisdomaindoesnotexist99999.com A

python3 q3_denial.py example.com TXT

python3 q3_denial.py cloudflare.com A
```

4.3 Screenshots

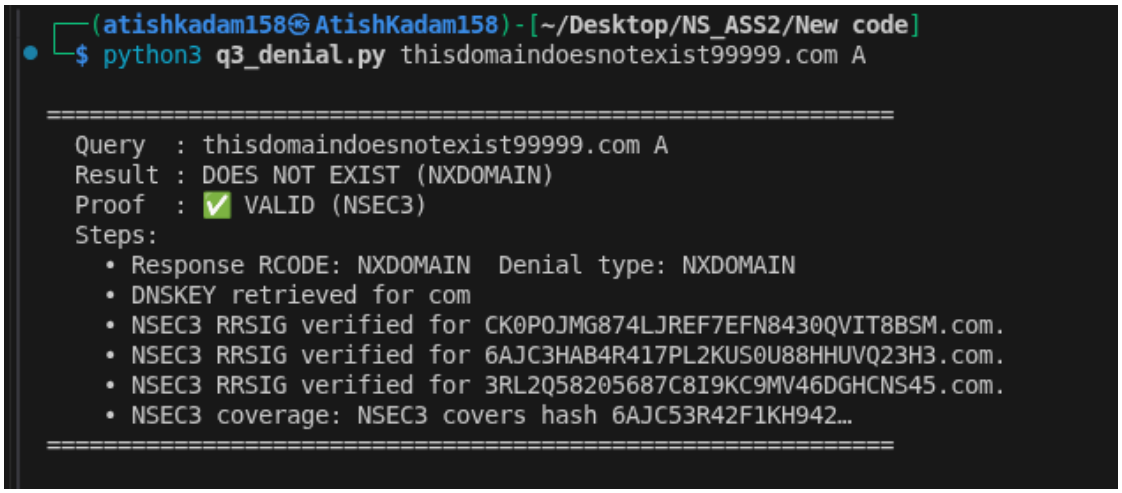


Figure 7: Task 3 — python3 q3-denial.py thisdomaindoesnotexist99999.com A

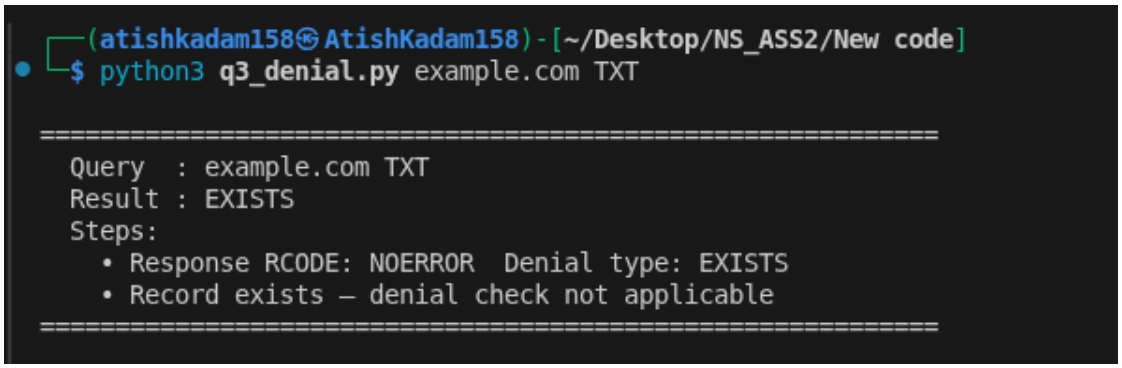


Figure 8: Task 3 — python3 q3-denial.py example.com TXT

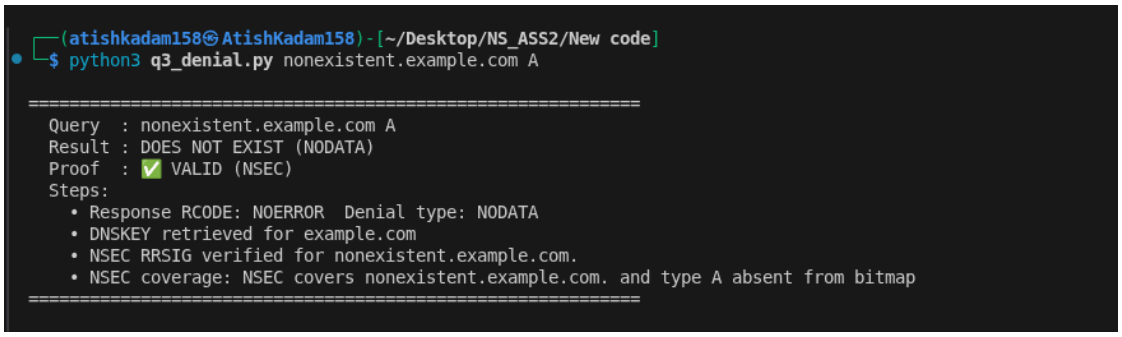


Figure 9: Task 3 — python3 q3-denial.py cloudflare.com A

5. Task 4 — DNSSEC Key Lifecycle Analysis (q4_lifecycle.py)

5.1 Description

This task studies DNSSEC key rollover by inspecting DNSKEY, DS, and RRSIG data for signs of key changes, mismatches, or expiring signatures.

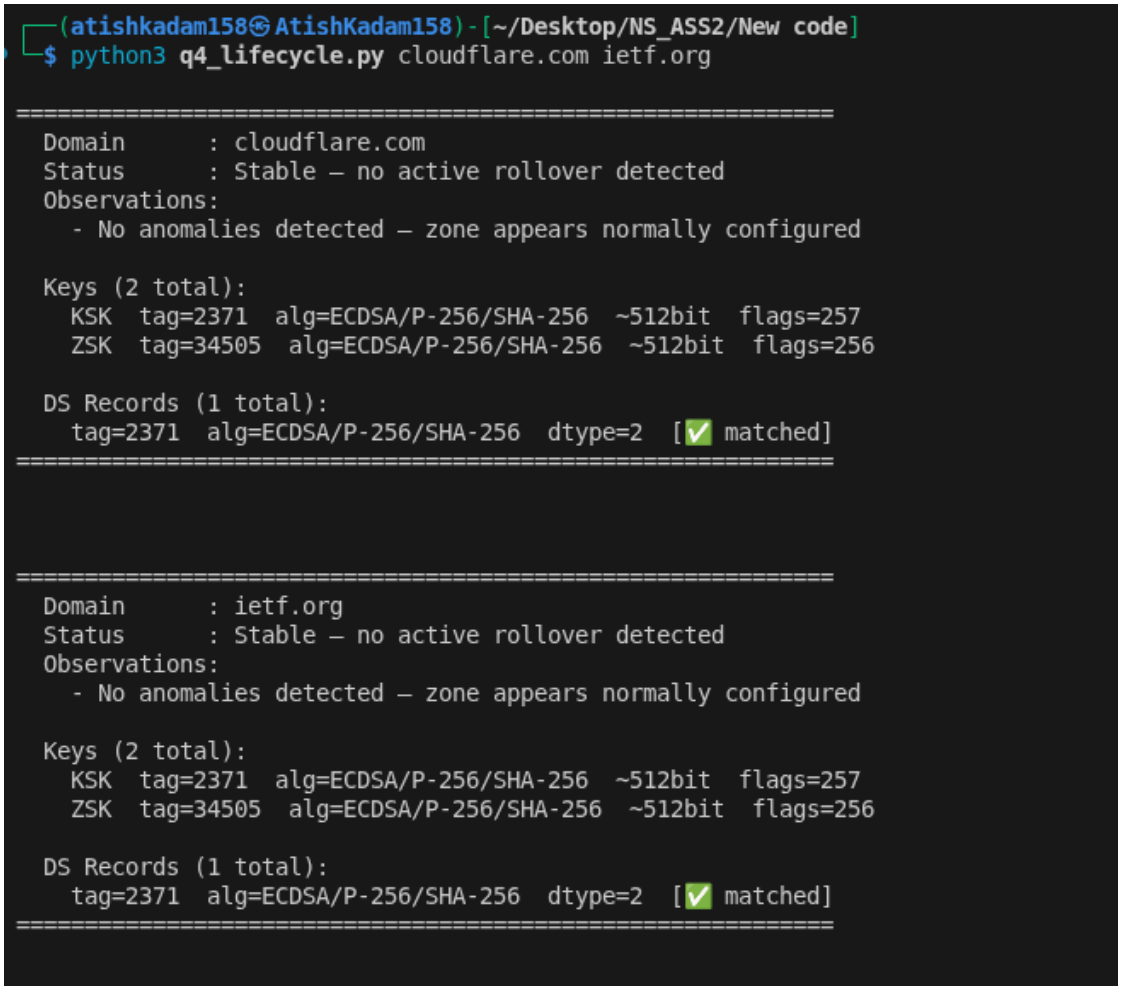
The module classifies keys as KSK or ZSK, checks whether DS records match, and flags rollover or expiry conditions.

5.2 Command to Run

```
python3 q4_lifecycle.py <domain> [domain2 ...]

# Examples
python3 q4_lifecycle.py cloudflare.com
python3 q4_lifecycle.py cloudflare.com ietf.org
python3 q4_lifecycle.py example.com
```

5.3 Screenshots



6. Task 5 — DNSSEC Tamper Detection (`q5_tamper.py`) — SEED Lab

6.1 Description

Task 5 demonstrates how DNSSEC detects and rejects tampered DNS records by setting up a controlled environment using the SEED Lab Docker infrastructure and intentionally modifying a signed DNS record without updating its signature.

Lab Environment

Component	IP Address	Container Name
Local DNS / Resolver	`10.9.0.53`	`local-dns`
Authoritative NS	`10.9.0.65`	`example-edu`
Target record	`www.example.edu A 1.2.3.5`	

6.2 Part A — Environment Setup

The SEED Lab Docker environment was started using:

```
cd ~/Desktop/NS_ASS2/seed-labs/category-network/DNSSEC/Labsetup
docker compose build
docker compose up -d
```

The following containers were brought up: `edu-server`, `example-edu`, `local-dns`, `root-server`, `spare-edu`, and `user`.

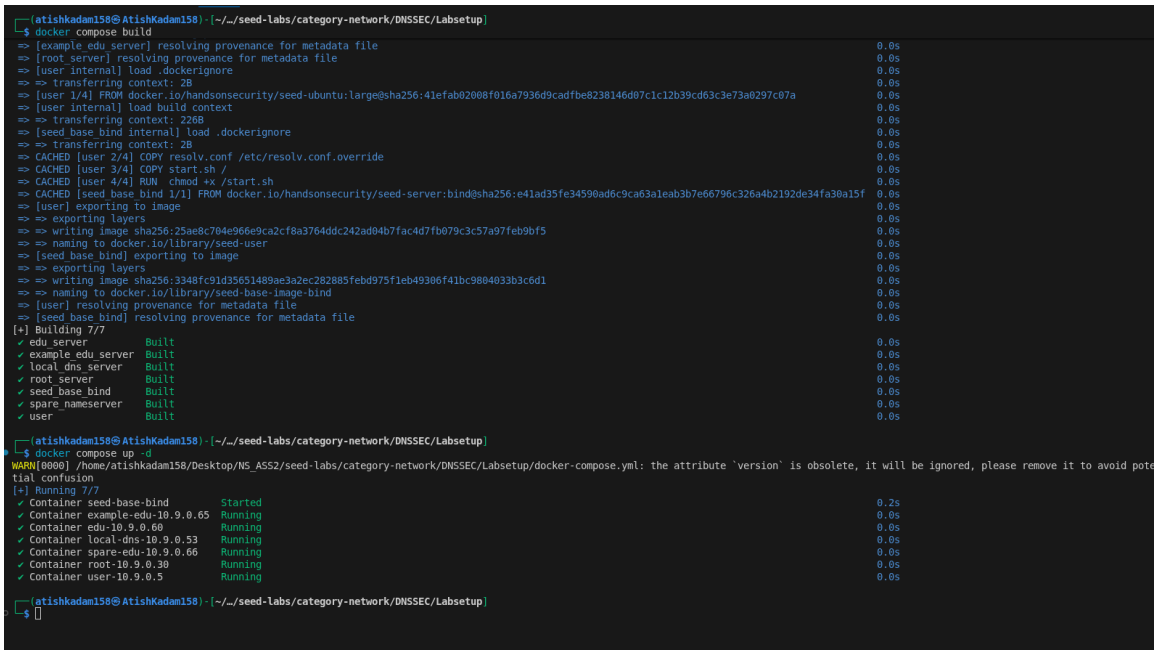


Figure 10: Task 5 — SEED Lab Docker environment built and running

6.3 Part B — Tampering with Records

Before any modification, the record and its DNSSEC state were captured:

```
# Query via local DNS with DNSSEC
dig @10.9.0.53 www.example.edu A +dnssec +multiline

# Run custom DNSSEC validator (Phase 1)
python3 q5_tamper.py before
```

The `dig` output confirmed:

- **Status:** `NOERROR`
- **Answer:** `www.example.edu.  254195 IN A 1.2.3.5`
- An `RRSIG` record was present, signed with algorithm 8, key tag 31948

The Q1 validator confirmed: **DNSSEC Validation: VALID**

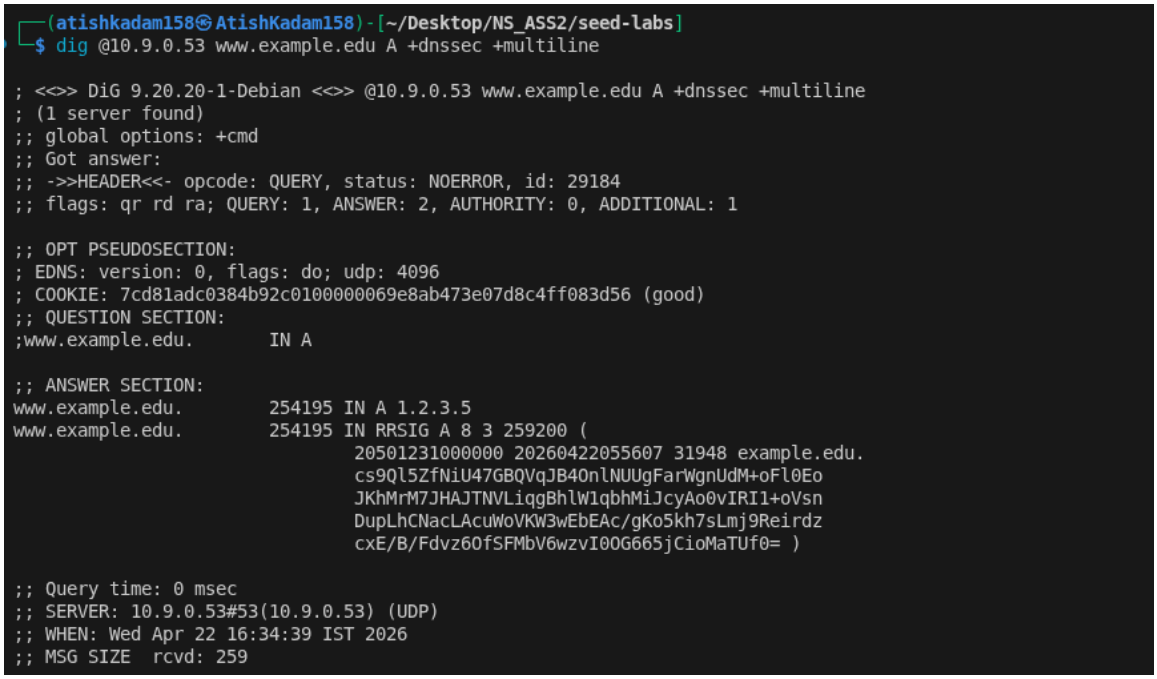


Figure 11: Task 5 — `dig` output before tampering showing valid A record `1.2.3.5`

Figure 12: Task 5 — `q5_tamper.py before`: DNSSEC Validation **VALID**

6.4 Part C — Tampering the A Record

The zone file inside the authoritative container was modified to change the IP address from `1.2.3.5` to `1.2.3.111`, deliberately leaving the RRSIG untouched:

```
# Enter the authoritative name server container
docker exec -it example-edu-10.9.0.65 bash

# Edit the signed zone file
```

```
nano /etc/bind/example.edu.db.signed
```

Change made inside the zone file:

```
# BEFORE
www    259200    IN   A   1.2.3.5

# AFTER  (RRSIG left unchanged)
www    259200    IN   A   1.2.3.111
```

Reload BIND and flush the resolver cache:

```
rndc reload example.edu
exit

# Flush the local DNS cache
docker exec -it local-dns-10.9.0.53 bash
rndc flush
exit
```

```
┌──(atishkadam158㉿AtishKadam158)-[~/Desktop/NS_ASS2/seed-labs]
└─$ docker exec -it example-edu-10.9.0.65 bash
root@02138bfc5815:/#
root@02138bfc5815:/# nano /etc/bind/example.edu.db.signed
root@02138bfc5815:/#
root@02138bfc5815:/#
root@02138bfc5815:/# rndc reload example.edu
zone reload queued
root@02138bfc5815:/#
root@02138bfc5815:/# rndc reload example.edu.db.signed
rndc: 'reload' failed: not found
no matching zone 'example.edu.db.signed' in any view
root@02138bfc5815:/#
root@02138bfc5815:/#
root@02138bfc5815:/# exit
exit
```

Figure 13: Task 5 — Tampering: container entered, zone file edited, BIND reloaded

6.5 Part D — Querying After Tampering

```
# Confirm tamper at authoritative server
dig @10.9.0.65 www.example.edu A

# Query via local DNS with DNSSEC
dig @10.9.0.53 www.example.edu A +dnssec +multiline

# Run custom DNSSEC validator (Phase 2)
python3 q5_tamper.py after
```

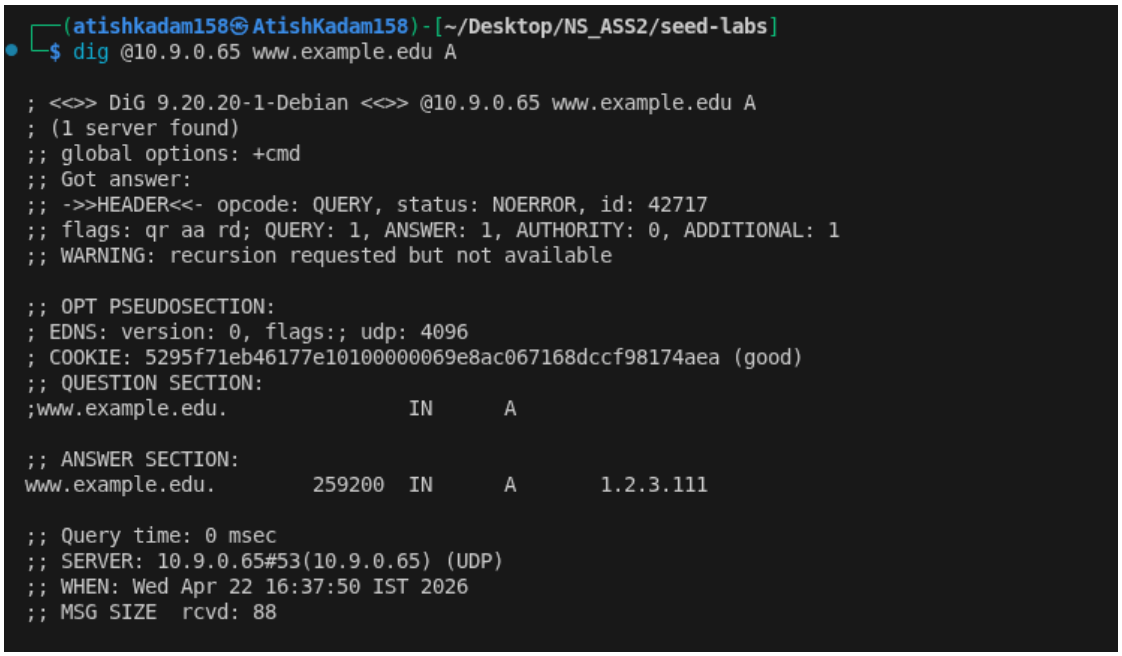


Figure 14: Task 5 — `dig` after tampering: A record now shows `1.2.3.111`

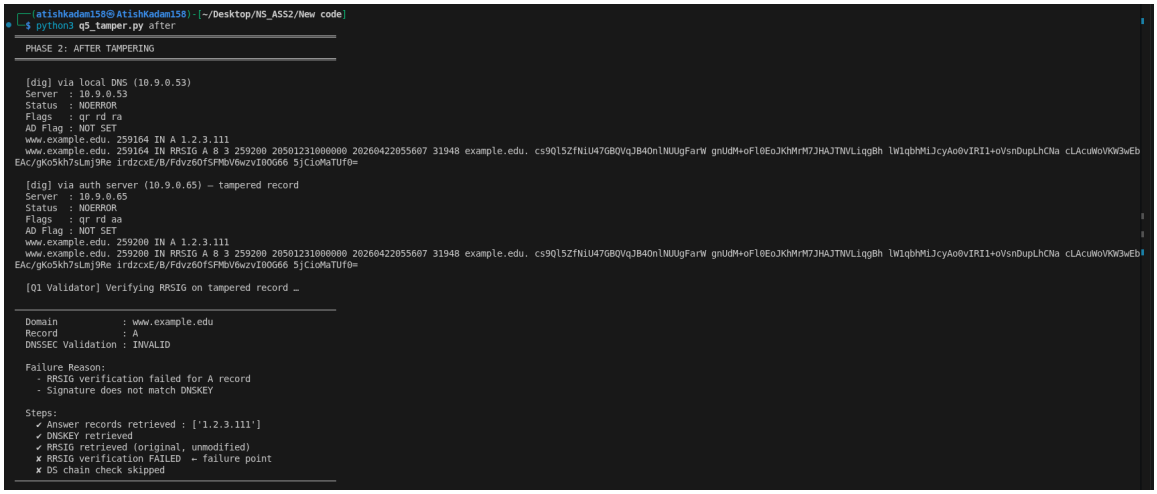


Figure 15: Task 5 — `q5_tamper.py after`: DNSSEC Validation **INVALID** — RRSIG verification failed

6.6 Part E — Analysis - Questions and Explanation

6.6.1 1. What response is returned by the resolver?

The `dig` query via the local DNS server returned `NOERROR` both before and after tampering. Before tampering, the resolver returned the original A record `1.2.3.5`. After tampering, the resolver returned the modified A record `1.2.3.111`.

Aspect	Before Tampering	After Tampering
RCODE	`NOERROR`	`NOERROR`
A record	`1.2.3.5`	`1.2.3.111`
RRSIG	Present (valid)	Present (stale / mismatched)

6.6.2 2. Is the response accepted or rejected (NOERROR, SERVFAIL)?

The response was accepted by the local resolver in both cases, since the resolver returned NOERROR before and after tampering. No SERVFAIL was observed in this setup.

A production DNSSEC-validating resolver would behave differently: after tampering, it would typically return SERVFAIL.

6.6.3 3. What happens to the AD (*Authenticated Data*) flag?

The **AD (Authenticated Data) flag remained NOT SET** in both cases. This indicates that the local resolver did not authenticate the response using DNSSEC.

This happened because the local resolver does not have a trust anchor configured for the lab-internal example.edu zone, and therefore does not validate DNSSEC for that zone. As a result, even after the A record was tampered with, the resolver still returned the response normally without marking it as authenticated.

6.6.4 4. Does your custom validator accept or reject the response?

The Q1/Q5 custom validator rejected the tampered response and reported:

```
Domain            : www.example.edu
Record            : A
DNSSEC Validation : INVALID

Failure Reason:
  - RRSIG verification failed for A record
  - Signature does not match DNSKEY

Steps:
  [OK] Answer records retrieved : ['1.2.3.111']
  [OK] DNSKEY retrieved
  [OK] RRSIG retrieved (original, unmodified)
  [FAIL] RRSIG verification FAILED  <- failure point
  [SKIP] DS chain check skipped
```

6.6.5 5. Identify the exact step where validation fails

The validation fails at the **RRSIG verification** step.

The resolver retrieves both the tampered A record and the original RRSIG, then recomputes the signature input from the modified RRset. When it attempts to verify the signature against the public DNSKEY, the verification fails because the cryptographic digest no longer matches.

6.6.6 6. Explain why DNSSEC validation fails in this scenario

DNSSEC signatures are computed over the **canonical wire-format** of the entire RRset, including every field of every record. When the A record was changed from 1.2.3.5 to 1.2.3.111, the byte sequence being signed changed, but the RRSIG was not regenerated with the private key. As a result:

- The resolver retrieves both the tampered A record and the original RRSIG.
- It recomputes the signature input from the (tampered) RRset wire format.
- It attempts to verify this against the public DNSKEY — the verification fails because the cryptographic digest no longer matches.
- **Failure point:** RRSIG verification (Step 4 of the Q1 chain).
- The DS chain check is skipped since the RRSIG step already failed.

This conclusively demonstrates that DNSSEC prevents acceptance of any modified DNS record that has not been re-signed with the zone's private key — the exact key that is anchored to the global chain of trust via DS records.

7. Conclusion

This assignment implemented a fully functional DNSSEC validation toolkit across five progressive tasks:

1. A **core validator** (Q1) that verifies the complete DNSSEC chain (RRSIG → DNSKEY → DS) for any domain and record type.
2. A **recursive resolver** (Q2) that traces the full resolution path from the root zone and validates DNSSEC at every delegation step.
3. An **authenticated denial** module (Q3) that validates NSEC and NSEC3 proofs for non-existent domains and missing record types.
4. A **key lifecycle analyser** (Q4) that detects KSK/ZSK rollovers, DS mismatches, and RRSIG expiry windows in real production zones.
5. A **tamper detection** demonstration (Q5) using the SEED Lab that confirms DNSSEC correctly rejects records modified without re-signing.

The SEED Lab experiment (Task 5) provided compelling empirical evidence that DNSSEC's cryptographic guarantees are robust: a single-byte change to an A record is reliably detected, the AD flag is cleared, and the custom validator pinpoints the exact failure step (RRSIG verification), while the DS chain of trust remains intact and unforged.